

Offensive / Bug-Bounty Testing Guide

The offensive toolkit wraps **Nmap** in an authorization-gated, audited UI for **legal, authorized security testing** — your own infrastructure, lab machines, or in-scope bug-bounty targets.



Rules of engagement (read first)

Only scan systems you own or are explicitly authorized to test.

Unauthorized port scanning and vulnerability scanning are illegal in many jurisdictions and violate most providers' terms of service.

Built-in guardrails:

- **Authorization checkbox** — you must tick “I confirm I am authorized to scan this target” for every scan (`offensive.require_authorization`).
- **Allow-list** — `offensive.target_allowlist` restricts scanning to specific CIDRs/hosts. Strongly recommended; an empty list means no restriction.
- **Deny-list** — `offensive.target_denylist` blocks targets unconditionally (e.g. cloud metadata `169.254.169.254`).
- **Audit log** — every scan (and every blocked attempt) is recorded in `logs/audit.log` with the operator, target, profile, and result.
- **Concurrency cap** — `offensive.max_concurrent_scans` prevents runaway load.

To disable the toolkit entirely, set `offensive.enabled: false` .

Scan profiles

Profile	Nmap flags	When to use
Host Discovery (ping sweep)	<code>-sn -T4</code>	Map live hosts in a range without port scanning.
Quick Port Scan (top 100)	<code>-F -T4</code>	Fast triage of common ports.
Full TCP Port Scan	<code>-p- -T4</code>	Every TCP port (slow, thorough).
Service & Version Detection	<code>-sV --version-intensity 5</code>	Identify services + versions on open ports.
OS & Service Detection	<code>-sV -O</code>	Fingerprint the OS (needs root).
Vulnerability Scan	<code>-sV --script vuln</code>	Run Nmap's NSE vuln scripts (noisy/intrusive).

You can also supply a custom **port spec** (e.g. `22,80,443` or `1-1000`).

Inputs are validated — only digits, commas, and dashes are accepted — and Nmap is invoked as an argument list (never a shell string), so there's no command injection.

 **Privileges:** OS detection and SYN-based scans need root. Run via the systemd service as root, or `sudo` , for full capability. Connect (`-sT`) scans and service detection work unprivileged.

Running a scan (UI)

1. Open the **Scanner** tab (only visible when `offensive.enabled`).
2. Enter a **target**: an IP, CIDR (`192.168.1.0/24`), or hostname.
3. Pick a **profile** (the description updates to explain it).
4. Optionally set **ports**.
5. Tick the **authorization** checkbox.
6. **Start Scan** — it runs asynchronously in the background.
7. Watch **Scan History**; click **View** when status is `completed` to see open ports, services/versions, OS guess, and any NSE vuln findings.

A safe public test target provided by the Nmap project is `scanme.nmap.org` (scanning is permitted there for learning — do not hammer it).

Running a scan (API)

Everything in the UI is backed by a JSON API (session-authenticated):

```
# log in (save the session cookie)
curl -c cookies.txt -X POST http://localhost:5000/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"<your-password>"}'

# list profiles
curl -b cookies.txt http://localhost:5000/api/scan/profiles

# start an authorized scan
curl -b cookies.txt -X POST http://localhost:5000/api/scan/start \
  -H 'Content-Type: application/json' \
  -d '{"target":"scanme.nmap.org","profile":"service_detect","authorized":true}'
# -> {"ok": true, "scan_id": "abc123..."}

# poll result
curl -b cookies.txt http://localhost:5000/api/scan/abc123...
```

A typical bug-bounty recon workflow

Always confirm the program **scope** first and add the in-scope ranges to `offensive.target_allowlist` so you can't accidentally scan out of scope.

1. **Scope it.** Read the program policy. Add allowed CIDRs/hosts to the allow-list; add anything explicitly out-of-scope to the deny-list.
2. **Discover hosts.** Run **Host Discovery** on the in-scope range to find live assets.
3. **Triage ports.** Run **Quick Port Scan** on live hosts to spot obvious services fast, then **Full TCP** on interesting hosts.
4. **Enumerate services.** Run **Service & Version Detection** to capture exact product/version strings — the basis for finding known CVEs.
5. **Light vuln pass.** Run the **Vulnerability Scan** profile (NSE `vuln`) on authorized hosts to surface common issues (misconfigs, weak TLS, known vulnerable versions).
6. **Pivot to manual testing.** Use the findings (open ports, versions, banners) to guide manual testing with your other tools (Burp Suite, nuclei, etc.).
7. **Document.** The dashboard keeps scan history + raw XML; pair it with your own notes for the report. Respect rate limits and program rules.

 This dashboard handles **network-layer recon & enumeration**. For web-app testing (the bulk of most bounties) combine it with dedicated tools like Burp Suite, ffuf, and nuclei.

Interpreting results

- **Open port + service/version** → search for known CVEs for that exact version. Outdated services are common bounty findings.
 - **Unexpected open ports** → possible shadow services / misconfiguration.
 - **NSE vuln output** → scripts like `ssl-*`, `http-*`, `smb-vuln-*` flag concrete issues; verify manually before reporting (NSE can false-positive).
 - **OS guess** → helps tailor follow-up testing.
-

Output & retention

- Parsed results and raw Nmap XML are kept in memory per scan and shown in the UI. Export/raw access is available via `GET /api/scan/<id>` (`raw` field).
- Every scan start/finish is in `logs/audit.log` for accountability.

Stay ethical, stay in scope, and keep good records. Happy (authorized) hunting. 🎯